

s1.09 — Fault-tolerant large-file upload — chunking / resume

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Part integrity and safe resume — IC3 baseline](#)
- [Completion, abort, and provider boundaries — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: retain the first-hand report attribution in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: one `PUT` with the whole file and a retry loop around it.
 - First break: a `10 GiB` transfer that fails at `99%` restarts from zero, and a retry after a lost response can duplicate or corrupt the object.
 - Second break: two devices resume the same upload, or an abort races a completion, and nobody owns the final answer.
 - Minimum primitives
 - Fixed parts with checksums
 - A persisted session
 - A frozen manifest
 - One conditional state transition that decides completion.
- 🗣️: "Chunking makes retries cheap; the real design is the state machine that decides which bytes become the object."
- Opening script
 - "I will treat this as a resumable multipart upload with an explicit completion state machine."
 - "First I confirm the file size, the failure model, and whether another device may resume."
 - "Then I size parts and concurrency from retry cost, and define an API where every step is idempotent."
 - "The deep dive is part identity and the completion-versus-abort race across the database and storage."
 - "I will close with failure recovery, cleanup, and where I would change the design under stronger requirements."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Part size and concurrency; API semantics
Architecture	17:00–30:00	Diagram, data ownership, one request trace
Deep dive	30:00–45:00	Part identity, resume, completion race
Operations and tradeoffs	45:00–50:00	Failure table, cleanup, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Maximum object size?	10 GiB	Fixes part count and metadata volume
Failure model?	Unreliable network, client restarts, lost responses	Forces idempotent parts and an authoritative state query
Can another authorized device resume?	Yes, same tenant and user	Resume needs file identity plus fresh authorization, not just an upload ID
Can a target object be overwritten?	Yes, with a version precondition	Publication needs compare-and-set on the logical key
Integrity contract?	Per-part checksum plus optional whole-file digest	Decides what completion verifies and how long it takes
Who serves the bytes?	Managed object storage; control API is separate	Keeps bytes off the API fleet; introduces provider reconciliation

- Functional scope
 - Split a file into independently retrievable parts.
 - Persist an upload session that survives client and server restarts.
 - Upload directly to storage through scoped authorization where supported.
 - Publish the completed object only after required validation.
 - Support cancellation and orphan cleanup.

Requirement	Target	Enforced by
Maximum file size	10 GiB	Session creation rejects larger declared sizes
Default part size	8 MiB, server-chosen	Session response fixes part size for the session
Client concurrency	4 parts in flight	Client semaphore; server per-session in-flight cap
Control API latency	p99 < 200 ms, excluding byte transfer	Metadata-only writes; no byte proxying
Control-plane availability	99.95%	Stateless API fleet; replicated metadata store
Session expiry	24 h of inactivity	Expiry reconciler transitions to ABORTED
Staging reclaim	Within 24 h after expiry	Idempotent cleanup worker with retry
Data-plane resilience	Existing grants permit up to 15 min of part uploads during API unavailability	Scoped expiring storage grants; new grants and final publication still need the API

- Invariants

- Tenant and user authorization applies to create, resume, parts, completion, and abort.
- A sealed part's number, bytes, and checksum form an immutable identity in this proposed service.
- A published object references one complete verified manifest.
- Completion and abort cannot both win for the same session generation.
- An ETag is an object or part identifier from the provider, not a universal whole-file SHA-256.
- Non-goals
 - Continuous append to an existing object.
 - Cross-tenant deduplication.
 - Inline content scanning on the upload path.

Capacity and consequence — IC3 baseline

- Parts
 - $10 \text{ GiB} / 8 \text{ MiB} = 1,280 \text{ parts}$.
 - At an illustrative independent 1% failure probability per attempt, expected retries are about $1,280 \times 0.01 / 0.99 \approx 12.9$.
 - Retransmission is about 103 MiB, rather than restarting the whole 10 GiB object.
 - Correlated network failure can make retry behavior much worse.
 - Decision: part size is the retry unit
 - 8 MiB bounds wasted retransmission below 1% of the object while keeping part count in the low thousands.
- Client memory
 - Four buffered parts require approximately 32 MiB before protocol buffers.
 - Streaming each part can reduce buffering further.
 - Decision: bound concurrency with a semaphore
 - Never enqueue all parts with their bytes attached.
- Throughput
 - At an assumed 100 MiB/s aggregate goodput, byte transfer takes 102.4 s.
 - Four parallel requests cannot exceed the client's link, storage, or CPU limits.
 - Hashing and verification time is additional unless pipelined.
 - Decision: hash each part while the next is in flight
 - Whole-file verification runs in the finalizer, not on the client's critical path.
- Metadata
 - 100,000 active maximal sessions imply up to 128 million part records.
 - At roughly 200 B per part row this is on the order of 25 GB of hot metadata.
 - Decision: shard part rows by upload_id
 - Enforce per-tenant active-session and staging-byte quotas.
 - Decision: expiration is part of capacity planning, not just housekeeping.

Architecture and flow — IC3 baseline

```

Client -> upload control API -> session / manifest DB
|
+-> scoped part upload -> private staging object storage
|
Complete request -> freeze manifest -> verifier / finalizer
|
| immutable final object/version
|
| publication metadata
|
Expiry reconciler -> abort state -> retryable staging cleanup
  
```

- Control plane owns session metadata and authorization.
 - It does not proxy file bytes by default.
 - It issues bounded access for the required session and part scope.
 - Long-lived storage credentials never go to the client.
 - Cost: the client now talks to two systems, so the design must reconcile their views.
 - Storage owns durable byte transfer.
 - A storage part acknowledgment and a session metadata update may be separate operations.
 - Reconcile them using stored checksums and provider list or status operations.
 - Cost: provider semantics for part replacement and completion constrain the service contract.
 - Finalizer owns completion.
 - It freezes the selected manifest.
 - It verifies bytes or relies only on an explicitly documented provider checksum contract.
 - It publishes the object through a recoverable state transition.
 - Cost: completion is asynchronous, so the API must expose an authoritative state query.
- Authenticate and create or reuse a session using a tenant-scoped idempotency key.
- Bind the session to destination, size, and declared file fingerprint.
- Upload missing parts with bounded concurrency and stable identities.
- Persist each verified part's size, checksum, and storage identifier.
- Freeze a manifest and transition from `UPLOADING` to `COMPLETING`.
- Verify total size, ordered parts, and required whole-file integrity.
- Commit the final object and persist its immutable version.
- Publish `COMMITTED` and return the same result on completion retries.

API and data model — IC3 baseline

Surface	Contract	Conflict/error behavior
POST <code>/v1/uploads</code>	Destination, size, fingerprint; returns session and chosen part size	Same idempotency key plus different payload returns <code>409</code>
PUT <code>/v1/uploads/{id}/parts/{n}</code>	Bounded bytes and declared checksum	Same sealed identity returns prior result; different bytes conflict
GET <code>/v1/uploads/{id}/parts</code>	Cursor page of verified parts	Resume uses all pages, not just the first
POST <code>/v1/uploads/{id}:complete</code>	Ordered part manifest and completion key	<code>202</code> while verifying; same manifest returns same result
GET <code>/v1/uploads/{id}</code>	State and immutable final object version	Authoritative answer after ambiguous completion timeout
DELETE <code>/v1/uploads/{id}</code>	Abort an incomplete session	<code>409</code> if completion already won; idempotent if already aborted

State	Key	Fields/invariant
Session	<code>(tenant_id, upload_id)</code>	Destination, expected size/hash, state, generation, expiry, version
Create dedup	<code>(tenant_id, idempotency_key)</code>	Request hash and upload ID; bounded retention at least the retry horizon
Part	<code>(upload_id, generation, part_number)</code>	Size, SHA-256 or negotiated checksum, provider ETag, verified state
Frozen manifest	<code>(upload_id, manifest_hash)</code>	Exact ordered part identities; immutable once completing
Final publication	<code>(tenant_id, object_key, object_version)</code>	Verified location, size, digest, source upload ID

- Reference DDL

- Keys and uniqueness match the state table above and the [companion schema](#).
- Staged part bytes and the published object live in object storage; SQL holds sessions, dedup, part identities, frozen manifests, and publication records.

```
CREATE TABLE upload_sessions (
  tenant_id text NOT NULL, upload_id text NOT NULL, object_key text NOT NULL,
  declared_bytes bigint NOT NULL CHECK (declared_bytes >= 0), declared_sha256 text NOT NULL,
  part_size_bytes bigint NOT NULL CHECK (part_size_bytes > 0),
  state text NOT NULL CHECK (state IN ('UPLOADING', 'COMPLETING', 'COMMITTED', 'ABORTED')),
  generation bigint NOT NULL CHECK (generation >= 1),
  frozen_manifest_hash text,           -- set on UPLOADING -> COMPLETING; never changes afterwards
  completion_fence text,              -- operation identity of the in-flight completion attempt
  expires_at timestamptz NOT NULL, version bigint NOT NULL,
  PRIMARY KEY (tenant_id, upload_id),
  CHECK (state IN ('UPLOADING', 'ABORTED') OR frozen_manifest_hash IS NOT NULL)
);
CREATE INDEX upload_sessions_expiry ON upload_sessions (expires_at)
  WHERE state IN ('UPLOADING', 'COMPLETING');
CREATE INDEX upload_sessions_cleanup ON upload_sessions (expires_at) WHERE state = 'ABORTED';
CREATE TABLE upload_create_dedup (
  tenant_id text NOT NULL, idempotency_key text NOT NULL,
  request_hash text NOT NULL, upload_id text NOT NULL,
  expires_at timestamptz NOT NULL,           -- at least the client retry horizon
  PRIMARY KEY (tenant_id, idempotency_key),
  FOREIGN KEY (tenant_id, upload_id) REFERENCES upload_sessions
);
CREATE INDEX upload_create_dedup_expiry ON upload_create_dedup (expires_at);
CREATE TABLE upload_parts (
  upload_id text NOT NULL, generation bigint NOT NULL,
  part_number integer NOT NULL CHECK (part_number > 0),
  tenant_id text NOT NULL,
  size_bytes bigint NOT NULL CHECK (size_bytes >= 0), checksum text NOT NULL,
  staging_ref text NOT NULL,                -- immutable attempt-specific staging key
  provider_etag text,                       -- opaque validator, not a digest
  verified boolean NOT NULL DEFAULT false,
  PRIMARY KEY (upload_id, generation, part_number),
  FOREIGN KEY (tenant_id, upload_id) REFERENCES upload_sessions
);
CREATE TABLE frozen_manifests (
  upload_id text NOT NULL, manifest_hash text NOT NULL,
  tenant_id text NOT NULL, generation bigint NOT NULL,
  part_identities jsonb NOT NULL,           -- ordered (part_number, checksum); immutable
  frozen_at timestamptz NOT NULL,
  PRIMARY KEY (upload_id, manifest_hash),
  FOREIGN KEY (tenant_id, upload_id) REFERENCES upload_sessions
);
CREATE TABLE object_versions (
  tenant_id text NOT NULL, object_key text NOT NULL, object_version text NOT NULL,
  location text NOT NULL, size_bytes bigint NOT NULL CHECK (size_bytes >= 0),
  sha256 text NOT NULL, source_upload_id text NOT NULL, published_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, object_key, object_version),
  UNIQUE (tenant_id, source_upload_id),     -- one session publishes at most one version
  FOREIGN KEY (tenant_id, source_upload_id) REFERENCES upload_sessions
);
```

- Not captured by the DDL
 - Staged bytes, the final object, and the provider's own multipart state (`ListParts`) live in object storage; scoped upload grants are signed tokens, not rows.
 - Deviation from the companion: states are `UPLOADING/COMPLETING/COMMITTED/ABORTED` rather than `OPEN/FINALIZING/FAILED`; the part key gains `generation` and drops `tenant_id` as the state table says; `frozen_manifests` and `object_versions` are added so the frozen manifest and final publication are rows rather than a text manifest.
- Proposed service conventions
 - Require parts numbered `1..N` for simple validation.

- All non-final parts have the negotiated size.
- The last part has the remaining size.
- Empty files use a documented zero-byte path.
- This is a service convention, not a universal multipart API rule.

Part integrity and safe resume — IC3 baseline

- Resume identity
 - Save the upload ID locally with size and file fingerprint.
 - Reauthorize before returning session state.
 - Verify the local file still matches the session.
 - List verified parts and upload only missing identities.
 - A path or filename alone is not an immutable file identity.
- Concurrent same-part upload
 - Avoid overwriting a shared staging key blindly.
 - Upload to immutable attempt-specific storage keys.
 - Conditionally publish the first valid part identity in metadata.
 - A duplicate with identical checksum reuses that identity.
 - A different checksum receives a conflict and its unused staging bytes are cleaned up.
- Late part writers
 - Seal the manifest before finalization.
 - Reject metadata publication once the session leaves `UPLOADING`.
 - Direct storage uploads may finish later, but cannot alter the frozen manifest.
 - Cleanup removes their unreferenced immutable bytes.
- Checksums
 - Verify each part at receipt or through a documented storage checksum result.
 - Do not trust a checksum supplied only by the client as proof that stored bytes match.
 - Hashing the concatenation of part digests is not the SHA-256 of concatenated file bytes.
 - A standard whole-file SHA-256 requires hashing the ordered bytes or an equivalent supported incremental computation.
 - A Merkle root is a different integrity contract and must be named as such.

Completion, abort, and provider boundaries — IC3 baseline

```
COMPLETE(session, manifest):
  transaction / compare-and-set:
    if COMMITTED and manifest matches: return prior object version
    if UPLOADING: validate part identities and freeze manifest
    otherwise require same in-progress manifest or return conflict
    transition to COMPLETING with operation identity
  outside transaction:
    verify and finalize immutable object using operation identity
  transaction:
    require current completion attempt fence and expected session version
    require state == COMPLETING and the same frozen manifest hash
    record verified immutable final object version
    transition COMPLETING -> COMMITTED
```

- Crash between storage commit and metadata commit
 - Retrying a provider commit may not be supported after the upload has disappeared.
 - Reconcile using the destination object or version and the operation identity.
 - Never infer failure solely because `ListParts` no longer returns an active upload.
 - Use a unique immutable final staging object per completion attempt when provider finalization can race.
 - A fenced metadata transaction selects the published object

- Losing artifacts remain unreferenced.
- Do not let a stale worker overwrite a shared final key before checking its fence.
- Abort race
 - A conditional transition selects abort or completion.
 - If completion already froze the manifest, baseline abort returns conflict.
 - The reaper follows the same state transition as an explicit abort.
 - Physical byte deletion happens asynchronously and is retried.
- Existing object overwrite
 - Use an expected version or precondition when publishing over a logical key.
 - Two concurrent completed uploads must not silently overwrite each other's intended version.
- OCI distinction
 - OCI permits noncontiguous part numbers and assembles them in ascending order.
 - Reusing a part number commits the last uploaded part for that number.
 - Commit uses part numbers and corresponding ETags.
 - Completed upload parts are no longer listable through the active-upload interface.
 - Our stricter immutable-part service must account for those provider semantics.
 - See [OCI multipart API behavior](#).
- Zero-copy caveat
 - A custom object store can publish a manifest over existing chunks.
 - A managed provider may assemble or copy internally.
 - Do not promise zero byte copying without that storage contract.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Part response lost	One part	Client timeout	Retry same identity within deadline	Reuse verified part or list authoritative state
Client crashes	One session	Inactivity	Staging remains private	Resume after identity and authorization checks
Local file changes	One resumed upload	Size/fingerprint mismatch	Reject resume	Start a new session
Complete response lost	One session	Ambiguous operation result	Poll current state	Reconcile final object before retrying provider commit
Abort races with completion	One session	Conditional state conflict	Exactly one logical transition wins	Cleanup only unreferenced bytes
Corrupt part	One part	Checksum mismatch	Reject without publishing	Retry verified bytes
Expired authorization	One client's parts	Auth failure	Stop affected uploads	Reauthorize scoped access; keep session identity
Cleanup worker unavailable	Staging capacity	Orphan bytes and age	Restrict new staging if quota reached	Resume idempotent cleanup
Metadata store unavailable	All new sessions and completions	Health check, error rate	In-flight part uploads continue on existing grants	Fail over replica; parts reconcile from storage listing

- Observe
 - Part retry rate and checksum failures.
 - Effective goodput by file-size cohort.
 - Oldest `COMPLETING` operation.
 - Unreferenced staging bytes.
 - Session quota rejections.
 - Completion reconciliation outcomes.
- Retry discipline

- Exponential backoff with jitter.
- Bounded attempts and total elapsed deadline.
- Respect server retry hints.
- Retry only transient transport or service errors.
- A checksum or authorization failure requires a corrective action, not infinite retries.
- Tenant isolation
 - Staging prefixes and grants are scoped per tenant and session.
 - Quotas cap active sessions and staging bytes per tenant.
 - A tenant's token receives 404 on another tenant's upload IDs, not 403, to avoid existence leaks.
- Rollout and rollback safety
 - Session and part schema changes follow expand, migrate, contract.
 - A new finalizer version must accept manifests frozen by the previous version.
 - Canary the API fleet
 - In-flight uploads must survive a deploy because grants do not depend on API health.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Single request	One whole-object transfer	Simple protocol	Whole-file retry cost	Small objects and reliable links
Fixed-size multipart	Independent numbered parts	Resume and bounded retry unit	Session metadata and cleanup	Baseline large unreliable uploads
Content-defined chunks	Boundaries depend on content	Reuses shifted content	CPU, indexing, privacy complexity	Repeated backup or sync workloads
Proxy all bytes	API relays upload content	Central validation point	API bandwidth and buffering cost	Policy requires inspection inline
Direct storage upload	Scoped client-to-storage transfer	Removes API byte bottleneck	Provider semantics and reconciliation	Baseline managed storage integration

- IC4 stretch — stronger publication
 - Upload to an immutable private version.
 - Scan or verify before updating the logical public pointer.
 - Readers see the prior version until publication wins its precondition.
 - This separates provider completion from application visibility.
- IC4 stretch — deduplication
 - Scope dedup by tenant unless cross-tenant information leakage is explicitly addressed.
 - Content existence and timing can reveal another user's data.
 - Reference counts and garbage collection require atomic ownership accounting.
- IC4 stretch — 10× load
 - Shard metadata by upload ID while retaining tenant indexes for quotas.
 - Keep bytes off the API fleet.
 - Bound checksum-worker capacity and finalization backlog.
 - Adjust part size using measured request overhead and retry cost.
- IC4 stretch — cost and blast radius
 - Staging bytes are the dominant cost under abandonment
 - Quota and expiry are cost controls, not only hygiene.
 - Cell the metadata store per region so a store failure strands one region's sessions, not all.
- Weakest assumption
 - Independent 1% part failure
 - A correlated outage multiplies retries and can exhaust the elapsed deadline.
 - Mitigation: circuit-break the client after consecutive failures and resume later from authoritative state.

- Over-engineering warning
 - Do not promise transparent cross-device resume from an upload ID without file identity and fresh authorization.
 - Do not build content-defined chunking for a one-shot upload service.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Size, failure model, resume scope, overwrite rule	Names non-goals and what cutting them buys	Negotiates scope instead of accepting it
Estimates	Part count, retry cost, metadata volume	Lets retry cost pick part size; names the correlated-failure case	A number changes a decision
Architecture	Control plane, storage, finalizer split with one trace	States the cost of each split and the reconciliation it creates	Every component has a mechanism and a price
Deep dive	Immutable part identity, frozen manifest, conditional completion	Provider semantics versus service contract; crash-after-commit reconciliation	Reasons across the database and storage boundary
Operations	Failure table, cleanup, observability	Rollout compatibility, cost controls, cell blast radius	Volunteers degradation and rollout before being asked
Wrap	Recap and tradeoff	Names the weakest assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Is the multipart ETag the whole-file hash?
 - **IC3 baseline**
 - No universal rule permits that assumption.
 - **IC4 stretch**
 - Distinguish provider identity, transport checksum, and application end-to-end digest.
- Q2: A part upload finishes after completion begins.
 - **IC3 baseline**
 - The frozen manifest excludes later part identity changes.
 - **IC4 stretch**
 - Immutable storage keys plus conditional metadata publication avoid provider overwrite races.
- Q3: The provider completed the object but our process crashed.
 - **IC3 baseline**
 - Reconcile the object or version before attempting another commit.
 - **IC4 stretch**
 - Private immutable staging makes this safe even with concurrent logical-key replacements.
- Q4: The interviewer now asks for continuous append to the same object.
 - **IC3 baseline**
 - Immutable multipart completion is no longer the right API contract.
 - **IC4 stretch**
 - Use a versioned segment manifest or append log with explicit reader consistency.
- Q5: Can completion validate a SHA-256 from only independent part SHA-256 values?
 - **IC3 baseline**
 - Not the standard hash of the complete file.
 - **IC4 stretch**
 - Negotiate a Merkle-tree contract or stream the full ordered bytes through verification.
- Q6: Does a cancelled upload release space immediately?

- **IC3 baseline**
 - Logical abort happens first
 - Physical cleanup is asynchronous.
- **IC4 stretch**
 - Include abandoned in-flight attempts and reconciliation failures in staging quotas.
- Q7 (premise reversal): Why not proxy all bytes through the API and skip provider reconciliation?
 - **IC3 baseline**
 - The API fleet becomes the bandwidth bottleneck and every deploy interrupts transfers.
 - **IC4 stretch**
 - Proxy only when inline inspection is mandatory, and then shard the proxy tier per tenant with its own capacity model.
- Validation scenarios
 - Lost part response with identical retry.
 - Conflicting bytes for the same part number.
 - Completion versus abort race.
 - File changed before resume.
 - Crash after provider commit but before database update.
 - Expired access token during upload.
 - Multi-page part listing.
 - Cleanup racing with a late storage write.

Recall card

- "Chunk and checksum make retries small and observable."
- "Resume requires both stable file identity and current authorization."
- "Freeze the manifest, verify integrity, then publish one immutable version."
- "Completion and cleanup are recoverable state machines across database and storage boundaries."
- Wrap-up script
 - "The design is fixed-size parts with immutable identities, a persisted session, and a frozen manifest."
 - "Completion is one conditional transition, so abort and complete cannot both win."
 - "The main tradeoff is direct-to-storage upload, which removes the API bottleneck but forces reconciliation with provider semantics."
 - "My weakest assumption is independent part failure; I would validate retry behavior under correlated outages first."

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [OCI multipart API](#): part ordering, replacement, commit, and listing behavior.
- [OCI multipart overview](#): retry and abandoned-upload cleanup.
- [Object-storage supporting design](#): retained background
 - The proposed upload contract above defines the authoritative s1 semantics.